PROJECT-BASED LEARNING

PHASE-II EVALUATION

Smart Music Shuffler: Fair, Repeat-Free Playlist Shuffling with Custom Data Structures in C++

Team ID: *DSCPP-III-2026-T142*

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: *DSCPP-III-2026-T142*
Semester: *3rd*
Section: *C, ML3, ML2, H*
Domain: *Data Structures and Programming in C++*

Team Members

1. *Bhuvi Singhal – 2027720 (Leader)*
2. *Akshat Joshi – 2028632*
3. *Akshank Chauhan – 2028630*
4. *Tansi Singh – 2028216*

Mentor

Ms. Nishtha Bhatt

Interactions completed: _____

Each member owns one data-structure module and presents it.

Phase-I Recap & Feedback Incorporated

Phase-I Status

Phase-I proposed a terminal music player built without STL: a circular doubly linked list, Fisher-Yates shuffle, open-addressing hash set and LIFO stack, with Win32 .wav playback. Mentor assessment: **Approved**.

Feedback Received

- *Mentor remark: "Add project architecture diagram or flowchart"*

Changes Incorporated

- *Architecture diagram added to the Phase-I report*
- *Diagram updated to match the implemented code (slide 5)*
- *Scope refined: non-repetition defined per playback cycle; fairness to be validated statistically*

Mentor feedback has been addressed, and the architecture now reflects the implemented system.

Problem Statement & Finalized Objectives

Problem Statement

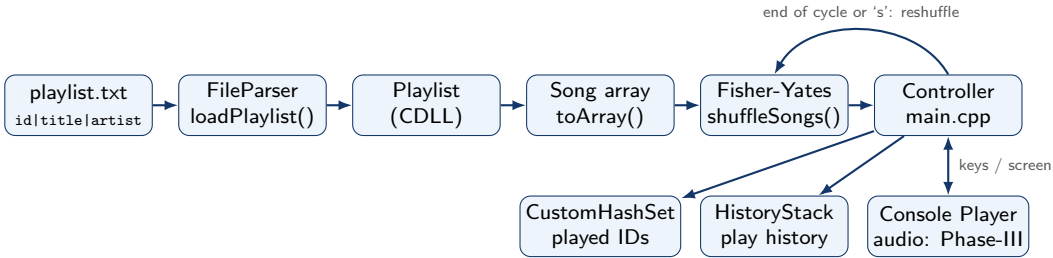
A basic shuffle picks a random index every time and keeps no memory, so songs repeat early. Simple swap-based shuffles can also favour some orders. We build a Windows console shuffler without STL, sized for a few hundred songs, that plays each song once per cycle using Fisher-Yates.

Finalized Objectives

1. *No song repeats during automatic forward playback within one cycle.*
2. *Shuffle with Fisher-Yates in $O(n)$ and check fairness with frequency and χ^2 tests.*
3. *Build list, hash set, stack and parser from scratch; verify memory safety with a sanitizer run.*
4. *Play local .wav files through the Windows API with SND_ASYNC (Phase-III).*

Objectives 1 to 3 are measurable now; objective 4 belongs to Phase-III.

Updated System Architecture / Workflow



Major Components

- *FileParser + Playlist: text file loaded into a CDLL*
- *Fisher-Yates: $O(n)$ in-place shuffle of the exported array*
- *Controller: updates hash set, history stack and console*

Changes Since Phase-I

- *New cycle starts at the end of the shuffled array, not when the set is full*
- *Controller module added between the shuffler and the data structures*
- *Audio simulated by a console timer until Phase-III*

Module Design & Functional Requirements

Major Modules

1. *FileParser + Playlist (CDLL)*
2. *FisherYates shuffle and test helpers*
3. *CustomHashSet + HistoryStack*
4. *Controller / menu (main.cpp)*

Functional Requirements

- *Load `id|title|artist` songs from playlist.txt*
- *Add, delete by ID and traverse the playlist*
- *Shuffle in $O(n)$; reshuffle when the cycle ends*
- *Record played IDs and history; search songs*

Module Dependency

Song (id, title, artist) is defined once in FisherYates.h and shared by Playlist and HistoryStack. FileParser fills Playlist. The controller uses all modules.

Technology Stack & Development Methodology

Technology Stack

- Programming: *Standard C++ (no STL containers)*
- Database: *None (flat file, playlist.txt)*
- Framework: *None (Win32 console API: windows.h, conio.h)*
- Development Tool: *GCC (MinGW), `g++ -std=c++17`*
- Version Control: *Git / GitHub*

Development Methodology

1. Requirement Analysis
2. System Design
3. Module Development
4. Integration
5. Testing & Validation

Plain C++ keeps the focus on pointers and memory, which is the goal of the course.

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C	*Circular doubly linked list, open-addressing hash set with linear probing and rehash, array-based stack, Fisher-Yates shuffle on arrays*	*Playlist, CustomHashSet, HistoryStack, FisherYates*
OOPs with C++	*Classes, encapsulation, constructors and destructors, Rule of Three, const getters*	*All classes*
Operating Systems	*Heap allocation and release; Win32 console API (cursor control, Sleep, keyboard polling). SND_ASYNC audio is planned*	*Controller (main.cpp)*
DBMS	*Not used; flat-file storage parsed into heap memory instead*	*FileParser*

Important: Mention only those concepts that have been **actually implemented** in the project.

Development & Implementation Progress

Module	Status	Work Completed
Playlist + FileParser	Completed	*CDLL add, delete by ID, display, array export; 100 songs loaded from playlist.txt*
FisherYates + CustomHashSet	Completed	*$O(n)$ in-place shuffle; linear probing, rehash before load exceeds 70%*
HistoryStack + Controller	In Progress	*Undo navigation complete; redo pending; controller implemented, repository integration pending*
Win32 audio + memory log	Pending	*PlaySound with SND_ASYNC; AddressSanitizer run and saved log*

Overall Project Completion: *About 60% (team estimate)*

Core data structures are complete; audio output and integration remain.

Working Prototype / Demonstration

Implemented Features

- *Menu: Live Player, All Songs, Search, Exit*
- *Shuffle on start; reshuffle when the cycle ends*
- *Next, previous, reshuffle, pause and resume*
- *History (last 10), Up Next (next 10), search and play*

Prototype / Output

```
NOW PLAYING
Blinding Lights - The Weeknd
0:42 [#######.....................]  2:50
track 1 of 100, played 1

n next | p previous | s reshuffle | space pause/play
h history | u up next | m menu
```

Playback is simulated by a timer; real audio is Phase-III.

All features run on the implemented data structures; audio output is simulated.

Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
Unit tests	*Parser loads 100 songs; links consistent after deletion; collisions resolved; LIFO order*	*All as expected*	Pass
Round-trip	*Each song ID appears exactly once after shuffle*	*All IDs preserved*	Pass
Fairness, memory	*Outcomes near ideal share; no leaks or invalid access*	*Not yet recorded*	Pending

Results / Validation

Focused module checks passed; they are not an exhaustive suite. Shuffle fairness (χ^2 test: 60,000 runs on 3 songs, 10,000 runs on 100 songs) and an AddressSanitizer memory log will be recorded in Phase-III.

GitHub Repository & Development Activity

Repository Information

- GitHub Link: `https://github.com/rixezz/Music-Shuffler`
- README.md: *Under revision; restored before final submission*
- Contributors: *Four contributors (all team members)*
- Repository Status: *Active; core modules committed*
- Branches: *main*

Development Activity

- *Commit history: 18 commits on main (2 Sep – 16 Oct 2026)*
- *Commit message conventions to be standardised in Phase-III*
- *Contributions by account: Akshat96af (11), bhuvisinghal08 (3), Artrisk1 (2), tansisingh3-lab (2)*
- *Core modules committed; controller follows integration testing*
- *Documentation: Phase-I report and presentation maintained in the repository*

Requirement: All team members must be contributors and must **commit individually using their own GitHub accounts**.

Individual Contribution

Member	Role	Technical Contribution	GitHub Commits
Bhuvi Singhal	*Lead, Playlist*	*Circular doubly linked list: add, delete by ID, display, toArray / fromArray; controller design*	*3*
Akshat Joshi	*Algorithm, Menu*	*Fisher-Yates shuffle and test helpers; FileParser; console menu and live player*	*11*
Akshank Chauhan	*Data Structure*	*CustomHashSet: linear probing, rehash at 70% load, copy constructor and assignment*	*2*
Tansi Singh	*History, Testing*	*HistoryStack: push, pop, peek, peekAt; redo navigation and integration tests planned*	*2*

Individual Evaluation

Every member should be prepared to explain and demonstrate their own technical contribution to the project.

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- *Technical: manual memory without STL; no safe copy in Playlist and HistoryStack*
- *Integration: backward navigation can replay a track; played-ID lookup not yet enforced*
- *Data / implementation: modulo bias in rand(); statistical results pending*

Solutions / Learning

- *Destructors and Rule of Three adopted; hasPlayed() check and history-based navigation planned*
- *Learning: passing tests show no detected faults, not proof of correctness*

Phase-III Roadmap

1. *Complete remaining modules: Win32 .wav audio, redo navigation*
2. *System integration: hasPlayed() check, navigation fix, controller commit*
3. *Testing / optimization: χ^2 tests, AddressSanitizer log*
4. *Final deployment / validation: executable with playlist.txt; build from a fresh clone*
5. *Documentation: README, final report and slides*

Expected Outcomes & References

Expected Final Outcomes

- *Working system: terminal player with real .wav playback*
- *Measurable results: fairness tests, cycle test, memory log*
- *Applicability: offline local player; teaching aid for fair shuffling*
- *Deployment: Windows .exe, sized for a few hundred songs*
- *Future scope: artist-aware spacing between consecutive tracks*

References

1. *R. Durstenfeld, "Algorithm 235: Random permutation", Communications of the ACM, 1964*
2. *L. Poláček, "How to Shuffle Songs?", Spotify Engineering blog*
3. *D.E. Knuth, The Art of Computer Programming, Vol. 2*
4. *Microsoft Learn: PlaySound function (mmsystem.h)*
5. *Project repository: github.com/rixezz/Music-Shuffler*

Thank You
Questions & Suggestions